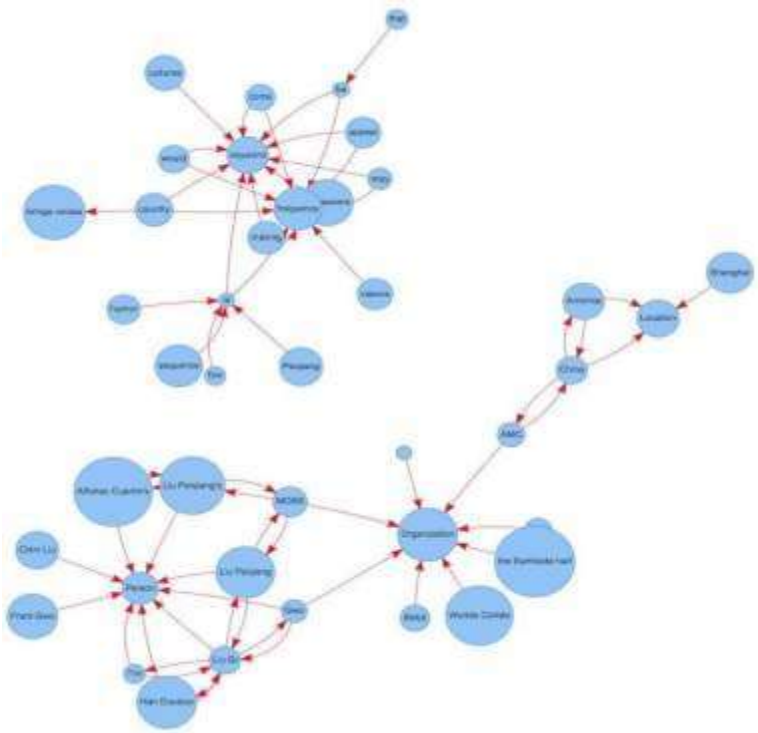
Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** TextRank for Document Summarization	
Experiment No: 10	**Date:**	**Enrolment No: 92301733056**

Aim: TextRank for Document Summarization

IDE: Google Colab

Theory:

TextRank is an algorithm based on PageRank, which often used in keyword extraction and text summarization. In this article, I will help you understand how TextRank works with a keyword extraction example and show the implementation by Python.



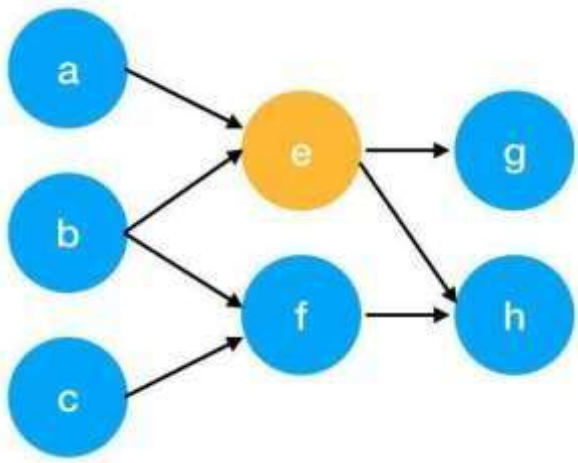
Understand PageRank

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** TextRank for Document Summarization	
Experiment No: 10	**Date:**	**Enrolment No: 92301733056**

There are tons of articles talking about PageRank, so I just give a brief introduction to PageRank. This will help us understand TextRank later because it is based on PageRank. PageRank (PR) is an algorithm used to calculate the weight for web pages. We can take all web pages as a big directed graph. In this graph, a node is a webpage. If webpage A has the link to web page B, it can be represented as a directed edge from A to B. After we construct the whole graph, we can assign weights for web pages by the following formula.

$$S(V_i) = (1-d) + d * \sum_{j \in In(v_i)} \frac{1}{|Out(V_j)|} S(V_j)$$

- S(Vi) - the weight of webpage i
- d - damping factor, in case of no outgoing links
- In(Vi) - inbound links of i, which is a set
- Out(Vj) - outgoing links of j, which is a set
- |Out(Vj)| - the number of outbound links



Here is an example to better understand the notation above. We have a graph to represent how web pages link to each other. Each node represents a webpage, and the

arrows represent edges. We want to get the weight of webpage e. We can rewrite the summation part in the above function to a simpler version.

$$In(v_e) = \{a, b\}, j \in \{a, b\}$$

$$\begin{aligned}\sum_{j \in \{a,b\}} \frac{1}{|Out(V_j)|} S(V_j) &= \frac{1}{|Out(V_a)|} S(V_a) + \frac{1}{|Out(V_b)|} S(V_b) \\ &= \frac{1}{|\{e\}|} S(V_a) + \frac{1}{|\{e, f\}|} S(V_b) \\ &= S(V_a) + \frac{1}{2} S(V_b)\end{aligned}$$

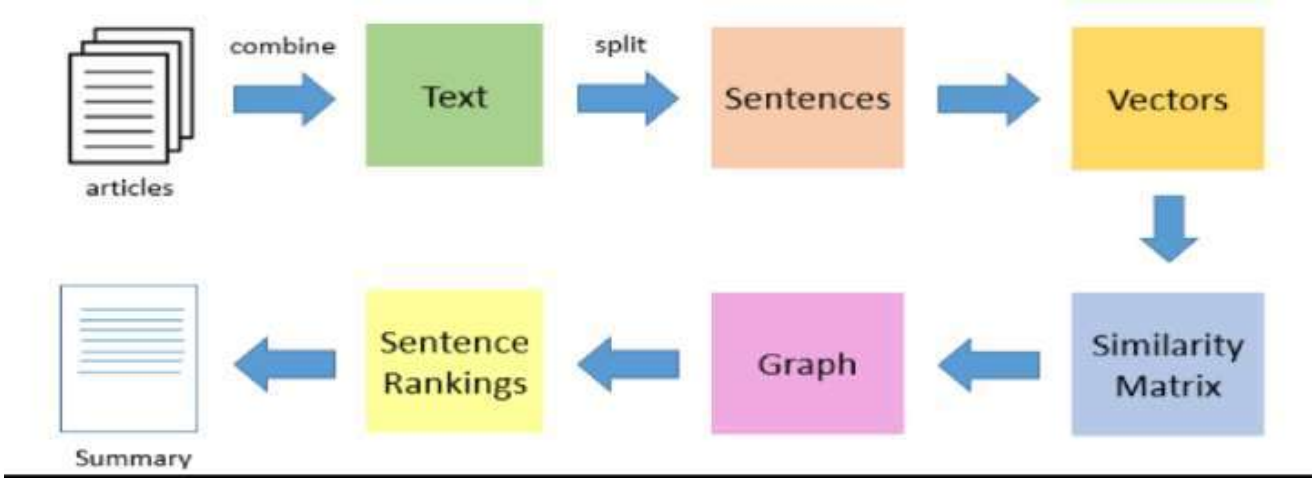
We can get the weight of webpage e by the following function.

$$S(V_e) = (1 - d) + d * \left(S(V_a) + \frac{1}{2} S(V_b) \right)$$

We can see the weight of the webpage e is dependent on the weights of its inbound pages. We need to run this iteration much time to get the final weight. In the initialization, the importance of each webpage is 1.

TextRank for document Summarization

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** TextRank for Document Summarization	
Experiment No: 10	**Date:**	**Enrolment No: 92301733056**



TextRank works in the following steps:

1. Tokenize documents into sentences.
2. Preprocess each sentence in the document.
3. Count key phrases and normalize them or produce TFIDF Matrix, you can also use any kind of vectorization such as spacy vectors.
4. Calculate the Jaccard Similarity between sentences and key phrases.
5. Rank the sentences with higher significance.

Pre Lab Exercise:

1. **How keyword extraction is similar to document summarization process?**

Keyword extraction and document summarization have quite a few things in common. First of all, their objective is to extract the most valuable information from a text. Both techniques go hand in hand with Natural Language Processing (NLP) as they analyze the frequency of the words, their relationships, and their importance. In fact,

both keyword extraction and document summarization often use ideas like TFIDF, graph-based ranking (TextRank), and similarity measures for figuring out the significance of the content. Besides, they both shrink the original text while keeping its main idea intact.

2. **How keyword extraction is different to document summarization process?**

One of the key differences is their output and intention. Keyword extraction comes up with a list of important words or phrases taken from the text while document summarization either picks pre-existing sentences or writes new ones in order to represent the text. For instance, keyword extraction may result in a list of terms like "machine learning NLP data" whereas the summary is a coherent paragraph. Summarization is a method of producing a readable and meaningful piece of writing that retains the original context, whereas keyword extraction is a method of identifying the most important words or phrases without the context.

Limitation of TextRank as a summarizer.

There are a few drawbacks to using TextRank for summarization. Firstly, it is an extractive technique, which means it can only pick out the sentences that already exist and is not capable of composing new ones. Secondly, since it ranks sentences by similarity, it can sometimes introduce redundant or repetitive sentences in the summary. Thirdly, the algorithm is not capable of deeply understanding the semantic meaning or context of the sentences, and as a result, it might miss out on major points. Furthermore, it tends to overlook less frequent sentences that are important, like conclusions. Lastly, the final quality of the summary is highly dependent on the preprocessing as well as the quality of the input text.

Code):

To be attached with

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** TextRank for Document Summarization	
Experiment No: 10	**Date:**	**Enrolment No: 92301733056**

```
# @title

#import basic libarires import nltk import numpy as np import networkx as nx

from nltk.tokenize import sent_tokenize, word_tokenize from nltk.corpus import
stopwords

from sklearn.metrics.pairwise import cosine_similarity from collections import Counter

# @title

#downloading nitk resources nltk.download('punkt') nltk.download('stopwords')
nltk.download('punkt_tab')

# @title def preprocessing(text):

 # First, let's make a list of common words we don't need to count, like 'the' or 'is'.

 stop_words=set(stopwords.words('english'))

 # Now, we break the big story (text) into smaller pieces, like individual sentences.
sentences=sent_tokenize(text)

 # We'll create an empty list to keep track of how many times each important word
appears in every sentence.   word_frequencies=[]

 # Let's go through each sentence, one by one.

 for sent in sentences:

   # We break each sentence into single words.     words=word_tokenize(sent)

   # We pick out only the real words (no numbers or symbols) and ignore the common
words from our list.     filtered_words=[word for word in words if word.isalnum()

                and word not in stop_words]

   # Then, we count how many times each of these important words shows up in this
sentence.     word_frequencies.append(Counter(filtered_words))

   # After checking the first sentence, we give back the sentences and our word counts
for this first sentence.     return sentences,word_frequencies
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** TextRank for Document Summarization	
Experiment No: 10	**Date:**	**Enrolment No: 92301733056**

@title def similarity_matrix(word_frequencies):

Imagine we have a list of 'word counts' for many sentences.

This 'size' tells us how many sentences we have. size=len(word_frequencies)

We're making a big grid (like a tic-tac-toe board) filled with zeros.

This grid will help us remember how similar each sentence is to every other sentence. sm=np.zeros((size,size))

Now, we take one sentence (let's call it 'i') and compare it to every other sentence (let's call them 'j'). for i in range(size): for j in range(size):

We don't want to compare a sentence to itself, because it's always perfectly similar! if i!=j:

We get the word counts for our first sentence.

words1=word_frequencies[i]

And we get the word counts for the second sentence we are comparing. words2=word_frequencies[j]

We find all the unique words that appear in *either* of these two sentences.

common_words=set(words1.keys()).union(set(words2.keys())) # Fixed .key() to .keys()

Now, we turn our word counts into special number lists (vectors) for sentence 'i'.

If a word isn't in a sentence, its count is 0.

vec1=np.array([words1.get(word, 0) for word in common_words]) # Use .get with default 0 # And we do the same for sentence 'j'.

vec2=np.array([words2.get(word, 0) for word in common_words]) # Use .get with default 0

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** TextRank for Document Summarization	
Experiment No: 10	**Date:**	**Enrolment No: 92301733056**

```
    # We ask the computer, 'How similar are these two number lists?'

    # The answer (a number between 0 and 1) tells us how much the sentences are talking about the same things.       sm[i][j]=cosine_similarity([vec1],[vec2])[0,0]

 # After checking every sentence against every other sentence, we give back our big grid of similarity scores.   return sm # @title def textrank_summarization(text,num_sentences=3):
sentences,word_frequencies=preprocessing(text)
sm=similarity_matrix(word_frequencies)

 #building graph   #print(sm)

 graph=nx.from_numpy_array(sm)   scores=nx.pagerank(graph)

 #print(scores)

 ranked_sentences = sorted(((scores.get(i, 0), s) for i, s in enumerate(sentences)), reverse=True)

 summary = " ".join([sent for _, sent in ranked_sentences[:num_sentences]])


 return summary

# @title

text="""As with other NLP tasks, text summarization requires text data first undergo preprocessing. This includes tokenization, stopword removal, and stemming or lemmatization in order to make the dataset readable by a machine learning model. After preprocessing, all extractive text summarization methods follow three general, independent steps: representation, sentence scoring, and sentence selection.
```

Representation

In the representation stage, an algorithm segments and represents preprocessed text data for comparison. Many of these representations build from bag of words models, which represent text segments—such as words or sentences—as datapoints in a vector

space. Large, multi-document datasets my use term frequencyinverse document frequency (TF-IDF), a variant of bag of words that weights each term to reflect its importance within a text set. Topic modeling tools such as latent semantic analysis (LSA) are another representation method that produce groups of summary keywords weighted across documents. Other algorithms, such as LexRank and TextRank, use graphs. These graph-based approaches represent sentences as nodes (or vertices) that are connected by lines according to semantic similarity scores. How do algorithms gauge semantic similarity?7

Sentence scoring

Sentence scoring, per its name, scores each sentence in a text according to their importance to that text. Different representations implement different scoring methods. For example, topic representation approaches score each sentence according to the degree that they individually express or combine key topics. More specifically, this may involve weighting sentences according to the co-frequency of topic keywords. Graph-based approaches, compute sentence centrality. These algorithms determine centrality using TF-IDF to calculate how far a given sentence node may be from a document's centroid in vector space.8

Sentence selection

The final general step in extractive algorithms is sentence selection. Having weighted sentences by importance, algorithms select the n most important sentences for a document or collection thereof. These sentences comprise the generated summary. But what if there is semantic and thematic overlap in these sentences? The sentence selection step aims to reduce redundancy in the final summaries. Maximal marginal relevance methods employ an iterative approach. Specifically, they recompute sentence importance scores in accordance with that sentence's similarity to already selected sentences. Global selection methods select a subset of the most importance sentences to maximize overall importance and reduce redundancy.9

As this overview illustrates, extractive text summarization is ultimately a text (and most often, sentence) ranking issue. Extractive text summarization techniques rank documents and their test strings (for sample, sentences) in order or produce a summary

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** TextRank for Document Summarization	
Experiment No: 10	**Date:**	**Enrolment No: 92301733056**

that best matches the central topics identified in the given texts. In this way, extractive summarization may be understood as a form of information retrieval.10

How abstractive text summarization works

As covered, abstractive text summarization techniques employ neural networks to generate original text that summarizes one or more documents. While there are numerous types of abstractive text summarization methods, literature does not use any one overarching classification system for describing these methods.11 Nevertheless, it is possible to overview the general aims of these various methods.""" # @title

textrank_summarization(text)

Results:

As with other NLP tasks, text summarization requires text data first undergo preprocessing. While there are numerous types of abstractive text summarization methods, literature does not use any one overarching classification system for describing these methods.11 Nevertheless, it is possible to overview the general aims of these various methods. Topic modeling tools such as latent semantic analysis (LSA) are another representation method that produce groups of summary keywords weighted across documents.

'

Observation:

Initially, it preprocesses the input text through NLTK, splitting the text into sentences and words, removing stopwords to keep only meaningful words. After that, each sentence is modeled as a word frequency vector through a counter-based approach. Once the preprocessing is done, the system builds a similarity matrix by measuring the cosine similarity between sentence vectors, which indicates how closely each sentence relates to the others. This similarity matrix helps to create a graph with NetworkX where sentences are the nodes and similarity scores are weighted edges connecting them.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** TextRank for Document Summarization	
Experiment No: 10	**Date:**	**Enrolment No: 92301733056**

The method proceeds by running the PageRank algorithm on the graph to see how important each sentence is depending on its links to the others. The sentences with higher scores are thought to be more relevant. At last, a few of the top scoring sentences are picked and merged to form a short summary that reflects the main points of the original text.

Post Lab Exercise:

1. Write about "your ownself" (in not more than 500 words→ You know better about you, rather than ChatGPT!!). Generate the summary of your portfolio in a. **50% of the size of your portfolio**

I am a dedicated ICT student with a strong interest in technology and problem-solving. I have worked on multiple projects such as a Library Management System, Smart Restaurant System, and Biometric Attendance System, gaining experience in PHP, MySQL, and web development.

I have also explored embedded systems using microcontrollers like ATmega32 and ESP32, working on hardware-based projects. Additionally, I have knowledge of machine learning concepts such as KNN and recommendation systems.

My strengths include continuous learning, logical thinking, and adaptability. I aim to become a skilled software developer and contribute to innovative and impactful projects.

b. **25% of the size of your portfolio**

I am an ICT student interested in software development and problem-solving. I have experience in web development, embedded systems, and basic machine learning. My goal is to become a skilled developer and work on innovative projects.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** TextRank for Document Summarization	
Experiment No: 10	**Date:**	**Enrolment No: 92301733056**

Comment over the summary obtained. Which sentence you think should be there in your summary, but was not spotted by textrank? Rate the overall summary in the scale of 1-5 (with 1 as least and 5 as highest rating). Paste the code, your portfolio, output and your analysis. Code:- summary = textrank_summarization(text)
print(summary)

Input Text:-

I am a passionate and dedicated student with a strong interest in technology, problem-solving, and innovation. Currently, I am pursuing my studies in Information and Communication Technology. I have worked on projects such as Library Management System, Smart Restaurant System, and Biometric Attendance System using PHP and MySQL. I have also done embedded systems using ATmega32 and ESP32. I have exposure to the basics of machine learning techniques such as KNN and recommendation systems.

Output:-

My strengths include problem-solving, adaptability, and continuous learning. In the future, I want to be a competent software developer and work on innovative projects. I am a kind of student who very much suka technology and solving problems. I have made projects like Library Management System, Smart Restaurant System, and Biometric Attendance System. I

have also experience with embedded systems and knowledge of machine learning techniques.

Analysis

The TextRank algorithm ranks sentences based on similarity and centrality in a graph structure. It effectively extracts sentences that are highly connected to others, which is why technical and project-related sentences are included in the summary. However, it does not consider the importance of unique or concluding statements, leading to the omission of career goals. This highlights a limitation of extractive summarization techniques, where contextual understanding is limited.